Занятие 1: Как работает веб-приложение

Клиент-серверная архитектура, HTTP, DNS

Слайд 1: Клиент и сервер

Клиент (Frontend)

- Браузер, мобильное приложение, desktop-приложение
- Отправляет HTTP-запросы
- Отображает пользовательский интерфейс
- Хранит состояние пользователя (cookies, local storage)

Сервер (Backend)

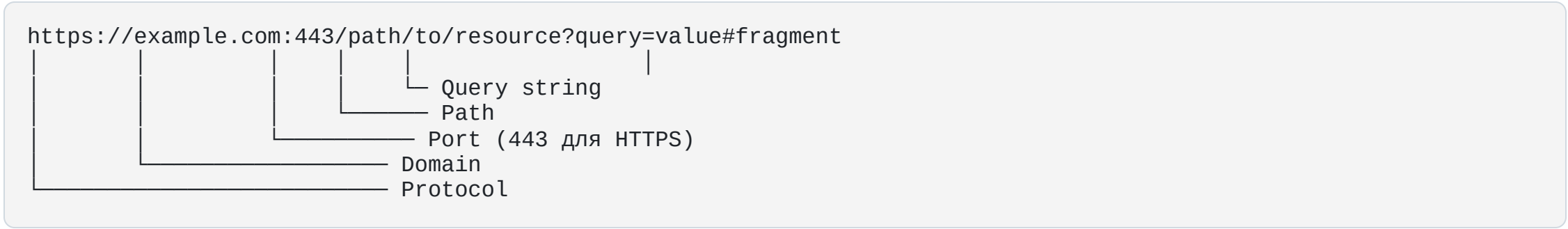
- Обрабатывает HTTP-запросы
- Выполняет бизнес-логику
- Взаимодействует с БД и внешними сервисами
- Возвращает HTTP-ответы

Распределение ответственности

- Frontend: отображение, взаимодействие с пользователем
- Backend: логика, данные, безопасность

Слайд 2: URL, DNS и IP-адрес

URL (Uniform Resource Locator)



DNS (Domain Name System)

- Преобразует доменное имя в IP-адрес
- Иерархическая система доменов
- Кэширование для ускорения

IP-адрес

- IPv4: 192.168.1.1
- IPv6: 2001:0db8:85a3::8a2e:0370:7334

Localhost

- 127.0.0.1 - локальный компьютер
- ::1 - IPv6 localhost

Слайд 3: Порты

Стандартные порты

- HTTP: 80
- HTTPS: 443
- SSH: 22
- FTP: 21

Порт компьютера vs порт контейнера

- Компьютер: физический или логический интерфейс
- Контейнер: изолированное сетевое пространство
- Проброс портов: `8080:80` - порт 8080 хоста → порт 80 контейнера

Localhost и порты

- http://localhost:8080 - локальный сервер на порту 8080
- http://127.0.0.1:3000 - то же самое через IP

Слайд 4: Структура HTTP-запроса

```
GET /api/users HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0
Accept: application/json
Authorization: Bearer token123
```

Компоненты:

- **Метод:** GET, POST, PUT, DELETE, PATCH
- **Путь:** /api/users
- **Версия протокола:** HTTP/1.1
- **Заголовки:** Host, User-Agent, Accept, Authorization
- **Тело:** (для POST, PUT, PATCH)

Слайд 5: Структура HTTP-ответа

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 123
Date: Wed, 10 Sep 2026 12:00:00 GMT

{
  "users": [
    {"id": 1, "name": "John"},
    {"id": 2, "name": "Jane"}
  ]
}
```

Компоненты:

- **Статус-код:** 200 OK
- **Заголовки:** Content-Type, Content-Length, Date
- **Тело:** JSON, HTML, XML, файл

Слайд 6: Основные статус-коды

2xx - Успех

- 200 OK - успешный запрос
- 201 Created - ресурс создан
- 204 No Content - успешный запрос без тела

3xx - Перенаправление

- 301 Moved Permanently - постоянный редирект
- 302 Found - временный редирект

4xx - Ошибка клиента

- 400 Bad Request - некорректный запрос
- 401 Unauthorized - требуется аутентификация
- 403 Forbidden - нет прав
- 404 Not Found - ресурс не найден

5xx - Ошибка сервера

- 500 Internal Server Error - внутренняя ошибка
- 502 Bad Gateway - ошибка шлюза
- 503 Service Unavailable - сервис недоступен

Слайд 7: Content-Type и форматы данных

Content-Type заголовок

```
Content-Type: application/json
Content-Type: text/html; charset=utf-8
Content-Type: application/xml
Content-Type: multipart/form-data
Content-Type: image/png
```

Форматы данных

- **HTML:** текст с разметкой для браузера
- **JSON:** структурированные данные для API
- **XML:** структурированные данные (устаревающий)
- **Формы:** application/x-www-form-urlencoded
- **Файлы:** multipart/form-data

Слайд 8: Роль компонентов веб-приложения

```
[Браузер]
    ↓ HTTP
[Веб-сервер: Nginx/Apache]
    ↓ (проксирование или WSGI/ASGI)
[Приложение: Django/FastAPI/Express]
    ↓ SQL
[База данных: PostgreSQL/MySQL]
```

Веб-сервер

- Приём HTTP-запросов
- Раздача статических файлов
- Балансировка нагрузки
- SSL/TLS завершение

Приложение

- Маршрутизация запросов
- Бизнес-логика
- Валидация данных
- Генерация ответов

База данных

- Хранение данных
- Запросы и транзакции

Слайд 9: Путь от ввода URL до ответа

1. Пользователь вводит URL в браузере
2. Браузер проверяет кэш DNS
3. DNS-запрос для получения IP-адреса
4. Установка TCP-соединения (handshake)
5. Для HTTPS: TLS handshake
6. Отправка HTTP-запроса
7. Веб-сервер принимает запрос
8. Передача приложению (если нужно)
9. Приложение обрабатывает запрос
10. Запрос к БД (если нужно)
11. Формирование HTTP-ответа
12. Возврат ответа браузеру
13. Браузер рендерит страницу

Слайд 10: Практическое задание

Задание 1: Анализ HTTP-запроса

Дан запрос:

```
POST /api/users HTTP/1.1
Host: api.example.com
Content-Type: application/json
Content-Length: 45

{"name": "John", "email": "john@example.com"}
```

Вопросы:

- Какой метод используется?
- На какой ресурс отправляется запрос?
- Какие заголовки указаны?
- Что передаётся в теле?

Задание 2: Выбор статус-кода

Для каждой операции выберите подходящий статус-код:

- Успешное получение списка
- Создание нового ресурса
- Ресурс не найден
- Некорректные данные в запросе
- Внутренняя ошибка сервера

Слайд 11: Вопрос для собеседования

Вопрос: Объясните путь от ввода адреса в браузере до получения ответа.

Ожидаемый ответ:

1. DNS-резолвинг
2. TCP-соединение
3. TLS handshake (если HTTPS)
4. HTTP-запрос
5. Обработка на сервере
6. HTTP-ответ
7. Рендеринг в браузере

Дополнительные вопросы:

- Что такое кэш DNS?
- Зачем нужен TCP handshake?
- В чём разница между HTTP и HTTPS?